

V2.0.0

CLAUDE CODE SMART ORCHESTRATOR

Put the right work on the right model.

A practical, auditable four-agent routing kit for planning, deep reasoning, fast implementation, and independent review.

PLAN

Architect

OPUS / HIGH

PERMISSION

PLAN

REASON

Deep Reasoner

OPUS / XHIGH

PERMISSION

PLAN

BUILD

Fast Worker

HAIKU / LOW

PERMISSION

DEFAULT

REVIEW

QA Reviewer

SONNET / HIGH

PERMISSION

DEFAULT

Configured targets are policy. Runtime traces are evidence. Verify both.

What v2 fixes

The first version described roles. Version 2 makes model routing explicit, tests the configuration defensively, and separates what is configured from what is actually observed at runtime.

1

Explicit model policy

Every agent declares a model alias, effort level, permission mode, and limited tool set.

2

Deterministic invocation

Use an explicit `@agent-name` mention when the test must call one named agent.

3

Honest runtime proof

Static validation proves files. A captured trace shows which agent and model were observed for one run.

4

Windows-native workflow

PowerShell instructions, safe backup steps, and both Bash and PowerShell tool availability where needed.

5

Stricter validation

Recursive scope checks, strict YAML, exact paths, policy enforcement, and secret-pattern scanning.

6

Deliberate permissions

Planning agents cannot edit. QA uses default mode so approved tests can run, while its prompt forbids intentional edits.

No magic and no guaranteed savings

Routing can improve the match between task and model. Actual quality, latency, and cost depend on prompts, project size, tool use, retries, provider behavior, and your plan. Measure your own workload.

Four lanes, one accountable loop

The coordinator remains responsible for decomposition, routing, context, acceptance criteria, and final integration. Agents do not recursively delegate.



Coordinator duties

- ☒ Choose the lane from task risk, not prestige.
- ☒ Send only the context the agent needs.
- ☒ Name allowed files and forbidden actions.
- ☒ Define evidence and a stop condition.
- ☒ Own integration and user-facing claims.

Hard boundaries

- Architect and Deep Reasoner are read-only planners.
- Fast Worker writes only inside the stated scope.
- QA does not intentionally edit, but a test command can mutate artifacts.
- QA shell access still requires user or session permission; host policy also matters.
- Agent output is evidence to inspect, not authority to trust blindly.

The configured targets

These aliases express policy without pinning dated model IDs. Claude Code resolves an alias at runtime, so record the observed model in your trace.

AGENT	MODEL	EFFORT	MODE	TOOLS	USE WHEN
architect	opus	high	plan	Read, Glob, Grep	Architecture, interfaces, sequence, tradeoffs.
deep-reasoner	opus	xhigh	plan	Read, Glob, Grep	Hard ambiguity, root cause, high-impact reasoning.
fast-worker	haiku	low	default	Read, Glob, Grep, Edit, Write, Bash, PowerShell	Clear, bounded implementation with named paths.
qa-reviewer	sonnet	high	default	Read, Glob, Grep, Bash, PowerShell	Independent review and exact user-provided tests.

Prefer the smallest sufficient lane

1. Is the task fully specified and low risk? Try Fast Worker.
2. Does it need structure or tradeoff analysis? Use Architect.
3. Is the uncertainty itself the problem? Use Deep Reasoner.
4. Is the work complete and ready to challenge? Use QA Reviewer.

Escalate on evidence

1. Stop a worker when scope is ambiguous.
2. Escalate when constraints conflict.
3. Do not turn retries into an unbounded loop.
4. Return to the coordinator for a new decision.

Configured target: the model alias and effort written in an agent file. **Observed route:** the agent and model reported by one captured runtime trace. Neither is a cryptographic guarantee about future runs.

What each lane should return

01 / ARCHITECT

OPUS

HIGH EFFORT

PLAN MODE

Return a decision-ready plan with boundaries, alternatives, risks, dependency order, and acceptance criteria. Do not edit files or run shell commands.

GOOD PACKET

"Design the auth boundary. Read only these three files. Compare two designs. Return a sequence and five acceptance checks."

02 / DEEP REASONER

OPUS

XHIGH EFFORT

PLAN MODE

Resolve the hardest uncertainty. State assumptions, test competing explanations, identify disconfirming evidence, and recommend the next decision.

GOOD PACKET

"Explain the intermittent race. Read the trace and scheduler only. Rank hypotheses and propose the smallest discriminating test."

03 / FAST WORKER

HAIKU

LOW EFFORT

DEFAULT MODE

Make the named change in allowed paths, run exact checks, summarize the diff, and stop on missing scope or a consequential decision.

GOOD PACKET

"Rename this flag in two files. Do not touch generated code. Run the named unit test. Report changed paths."

04 / QA REVIEWER

SONNET

HIGH EFFORT

DEFAULT MODE

Inspect independently. Run only the exact command if user or session permission allows. Do not intentionally edit; note that test commands can mutate artifacts.

GOOD PACKET

"Review this diff against three criteria. Run exactly: python -m unittest -q. Do not edit. Report denied or unavailable tools."

Why effort differs: effort is a policy signal, not a quality promise. Low effort suits bounded execution. High and xhigh are reserved for planning, ambiguity, and adversarial review where additional reasoning can justify the latency.

Deterministic invocation vs automatic routing

TEST THE NAMED AGENT

Use an explicit @mention

When the question is "does this exact agent work?", explicitly invoke it in Claude Code. Natural-language hints may be ignored or routed differently.

```
> Use @agent-architect to inspect the project structure. Return three risks. Do not edit.
```

-  Named agent tool call appears.
-  Agent result completes successfully.
-  Observed model matches the expected family.
-  No per-call model override changed the route.

EVALUATE THE ROUTER

Use unlabeled real tasks

When the question is "does the coordinator choose well?", omit the agent name. Run a labeled evaluation set and compare the observed choice to your expected lane.

```
> Propose two architectures for durable job retries. Compare failure modes. Do not edit.
```

- Define the expected lane before the run.
- Repeat to measure route stability.
- Score both routing and answer quality.
- Record non-routing, timeout, and retry cases.

QUESTION	CORRECT TEST	USEFUL EVIDENCE	NOT SUFFICIENT
Is the file valid?	Static validator	Strict schema and policy pass	Agent prose alone
Can this agent run?	Explicit @mention	Successful Agent call and result	"Please use..." prose
Which model ran?	Stream JSON trace	Assistant model event associated with the agent	Configured alias alone
Does the router choose well?	Blind task suite	Route accuracy, quality, latency, failure rate	One cherry-picked demo

Measure distribution on a real project

A router is useful only when it sends representative work to the right lane and the integrated result holds up. Use a pilot that requires architecture, implementation, and review.

12+
Tasks

At least three representative tasks per expected lane.

3x
Repeats

Repeat automatic prompts to detect route instability.

2
Scores

Score route choice separately from answer quality.

METRIC	DEFINITION	RECORD
Route accuracy	Observed lane matches the label assigned before execution.	Correct / total
Completion quality	Acceptance criteria passed without coordinator repair.	0-3 rubric
Boundary compliance	Agent stayed inside paths, tools, and stop conditions.	Pass / fail
Latency	Wall time from call to completed agent result.	Seconds
Failure behavior	Denied tools, malformed output, timeout, or unbounded retry.	Category
Usage evidence	Provider-reported usage or plan-level observations, if available.	Raw record

Pilot shape

1. Architect proposes the system plan.
2. Deep Reasoner resolves the hardest uncertainty.
3. Fast Worker implements bounded slices.
4. QA Reviewer challenges each completed slice.
5. Coordinator integrates and records outcomes.

Interpret carefully

- Do not infer savings from model labels.
- Do not compare unequal prompts or acceptance bars.
- Separate tool failures from model failures.
- Securely retain only the evidence needed for rechecking.
- Publish limitations with any benchmark.

Merge safely on Windows

The starter is project-scoped. Back up existing files, compare before copying, and preserve project-specific instructions. Do not replace a working `CLAUDE.md` blindly.

1. Validate the kit

```
PS> python -I .\starter\scripts\validate_kit.py
```

2. Back up project configuration

```
PS> $stamp = Get-Date -Format yyyyMMdd-HH:mm:ss
PS> Copy-Item .\claude ".\claude.backup-$stamp" -Recurse -ErrorAction SilentlyContinue
PS> Copy-Item .\CLAUDE.md ".\CLAUDE.md.backup-$stamp" -ErrorAction SilentlyContinue
```

3. Merge agents

```
PS> New-Item .\claude\agents -ItemType Directory -Force
PS> Copy-Item .\starter\claude\agents\*.md .\claude\agents\
```

4. Merge guidance

Read both `CLAUDE.md` files. Copy only the routing sections that fit the project. Existing security, build, and test instructions remain authoritative.

5. Review before running

```
PS> Get-Childitem .\claude\agents
PS> git diff -- .claude CLAUDE.md
```

6. Uninstall or roll back

Restore the timestamped backup, or remove only the four shipped agent files. Never recursively delete a computed path without checking the resolved target.

Permission reminder

Frontmatter is one layer. Claude Code permission settings, enterprise policy, hooks, shell behavior, and the operating system can be stricter or interact in surprising ways. Test in a disposable repository first.

PowerShell preview: with Git Bash installed, set
`$env:CLAUDE_CODE_USE_POWERSHELL_TOOL = '1'` if the tool is unavailable. Profiles are not loaded and Windows sandboxing is not provided; confirm current official tool guidance.

Give every agent a bounded contract

Better routing cannot rescue a vague handoff. A compact packet reduces context waste and makes success auditable.

Packet template

GOAL

One observable outcome.

CONTEXT

Only facts and files needed.

ALLOWED SCOPE

Exact paths and permitted actions.

CONSTRAINTS

Compatibility, security, time, style.

EVIDENCE

Exact checks or review criteria.

RETURN

Required output format.

STOP

Conditions that require escalation.

Architect packet

"Design a migration from local JSON to SQLite. Read three named modules. Preserve the public API. Return two alternatives, a sequence, risks, and acceptance checks. Do not edit."

Fast Worker packet

"Implement step one in `src/store.py` and its tests only. No dependency changes. Run the exact unit-test command. Stop if the existing interface is inconsistent."

QA packet

"Review the resulting diff for data loss, compatibility, and test coverage. Run exactly the provided command if allowed. Do not edit. Return severity, path, and evidence."

Anti-pattern: "Build the whole thing and make it production ready." It hides decisions, creates an unbounded write surface, and makes failures hard to attribute. Split the work until every packet has a visible finish line.

Capture what actually ran

A frontmatter file can be correct while a run is misrouted, overridden, malformed, or incomplete. Capture stream JSON for a small explicit probe and validate the evidence separately.

Evidence ladder

LEVEL 1

Static policy

Exact files, strict YAML, allowed fields, expected model and effort, permitted tools.

LEVEL 2

Invocation evidence

An Agent tool call names the expected subagent and completes successfully.

LEVEL 3

Model evidence

The associated assistant event reports the expected model family or alias.

LEVEL 4

Outcome evidence

The work passes its acceptance checks and respects boundaries.

Capture an explicit probe

```
PS> $trace = Join-Path $env:TEMP "ccso-architect-proof.jsonl"
PS> claude -p "Use @agent-architect to inspect the project. Return one risk. Do not edit." `
--output-format stream-json `
--verbose `
--no-session-persistence |
Set-Content -LiteralPath $trace -Encoding utf8
```

Then verify

- ☒ The JSONL is complete and parseable.
- ☒ Exactly the expected agent was invoked.
- ☒ One completed lifecycle links to a non-error Agent result.
- ☒ The observed model matches the expected family.
- ☒ No ambiguous second invocation confuses attribution.

Observed, not proven forever. A trace describes one execution in one environment. It does not attest effort. Keep traces outside the repository, redact before sharing, and never commit them.

Fail closed before you trust the kit

Agent files are executable configuration and prompt material. Review them like code. The v2 validator checks the packaged policy recursively instead of assuming every relevant file is at the top level.

Static validation gates

-  Expected agent paths exist exactly once.
-  No shadow agent appears in nested scope.
-  YAML rejects duplicate keys and wrong types.
-  Unknown frontmatter fields fail validation.
-  Model, effort, mode, and tools match policy.
-  Secret-like content is scanned recursively.
-  Unexpected executable hooks are rejected.

Runtime controls still matter

- Review global, project, and managed settings.
- Confirm user or session permission before QA shell tests.
- Treat test commands as potentially mutating artifacts.
- Keep secrets out of prompts, traces, and repositories.
- Use disposable repositories for first runs.
- Inspect every write before committing.
- Rotate any credential exposed in logs or transcripts.

Run the local gates

```
PS> python -I .\starter\scripts\validate_kit.py
PS> python -m unittest discover -s .\starter\tests -v
PS> git diff --check
```

THREAT	CONTROL	RESIDUAL RISK
Shadow agent file	Recursive exact-path allowlist	Other config scopes outside the kit
Malformed or duplicate YAML	Strict typed loader	Runtime schema changes
Unexpected shell behavior	Minimal tools and exact commands	Host policy and command side effects
Secret in archive	Recursive pattern scan	Novel formats and false negatives
Misleading routing claim	Static plus observed evidence labels	Future runtime variation

Build a real project in reviewable slices

Use the kit on work that exercises every lane. A planning-only demo cannot validate implementation distribution or the handoff between worker and reviewer.

PHASE A

System framing

Architect defines boundaries, data flow, delivery slices, and acceptance criteria. Coordinator approves or revises.

PHASE B

Uncertainty burn-down

Deep Reasoner investigates the hardest open question and proposes a discriminating experiment.

PHASE C

Bounded construction

Fast Worker implements one narrow slice with named paths and an exact check.

PHASE D

Independent challenge

QA Reviewer tests the slice and reports evidence without intentional edits; test commands may still mutate artifacts.

Coordinator log

FIELD	EXAMPLE
Task ID	API-04
Expected route	Fast Worker
Observed agent	fast-worker
Observed model	Recorded from trace
Acceptance	4 of 4 passed
Boundary	Pass
Retry	None
Notes	One denied tool, no impact

Ship criterion

The integrated project, not a single agent response, must pass the project test suite and human review. Router metrics explain the process; they do not replace product quality.

Stop criterion

Pause if agents repeatedly misroute, cross write boundaries, produce untraceable results, or need coordinator repair that outweighs the value of delegation.

When the route does not behave

SYMPTOM	LIKELY CAUSE	NEXT CHECK
Named agent never runs	Natural-language request, discovery scope, or malformed frontmatter.	Use an explicit @mention; run strict validation; inspect project scope.
Wrong model appears	Alias resolution, per-call override, settings, or stale agent definition.	Inspect the Agent input and assistant model events; confirm exact file loaded.
QA command is denied	User, session, or higher-priority permission policy blocks shell use.	Report the denial. Do not retry with command variants. Run the check in the coordinator if authorized.
PowerShell tool unavailable	CLI version, platform, or agent tool declaration mismatch.	Check Claude Code version and official PowerShell tool docs.
Worker edits too much	Packet did not name allowed paths or stop conditions.	Revert only with authorization; narrow the packet and rerun in a disposable branch.
Validator passes a rogue file	Old validator or wrong working directory.	Use v2, confirm recursive scan root, and run adversarial fixtures.
Trace verifier is inconclusive	Malformed, incomplete, or multiple-agent trace.	Fail closed and capture a fresh one-agent probe.
Results vary run to run	Automatic routing is probabilistic or task label is ambiguous.	Repeat the evaluation; improve task taxonomy; use @mention for deterministic agent tests.

Windows checklist

- ☒ Run from the intended project root.
- ☒ Use quoted paths when folders contain spaces.
- ☒ Prefer PowerShell-native file operations.
- ☒ Inspect resolved targets before recursive actions.
- ☒ Record CLI and kit versions with evidence.

A clean bug report includes

- Minimal redacted reproduction.
- Expected and observed agent.
- Expected and observed model family.
- Claude Code and kit versions.
- Static validator output and trace-verifier outcome.

Launch checklist, docs, and credits

Before the first run

- ☒ Read README-FIRST.md and starter/SETUP.md.
- ☒ Validate the untouched kit.
- ☒ Back up existing project instructions.
- ☒ Merge agent files and guidance deliberately.
- ☒ Run explicit one-agent probes.
- ☒ Review traces for observed routing.
- ☒ Start the real-project evaluation log.
- ☒ Keep claims limited to collected evidence.

Visual guide status

This guide explains the policy and workflow. It is not runtime evidence. Use the shipped validator, trace verifier, and your own project acceptance tests.

License

MIT License. See the packaged LICENSE file for the complete terms.

Credits

Built and maintained by Jeff J Hunter.

With appreciation to Matt Farmer for the original model-routing idea and public write-up that inspired this direction.

Official Claude Code references

- [Subagents overview](https://code.claude.com/docs/en/sub-agents)
<https://code.claude.com/docs/en/sub-agents>
- [Choose a model](https://code.claude.com/docs/en/sub-agents#choose-a-model)
<https://code.claude.com/docs/en/sub-agents#choose-a-model>
- [Invoke subagents explicitly](https://code.claude.com/docs/en/sub-agents#invoke-subagents-explicitly)
<https://code.claude.com/docs/en/sub-agents#invoke-subagents-explicitly>
- [Permission modes](https://code.claude.com/docs/en/sub-agents#permission-modes)
<https://code.claude.com/docs/en/sub-agents#permission-modes>
- [Model configuration and overrides](https://code.claude.com/docs/en/model-config)
<https://code.claude.com/docs/en/model-config>
- [PowerShell tool](https://code.claude.com/docs/en/tools-reference#powershell-tool)
<https://code.claude.com/docs/en/tools-reference#powershell-tool>

Inspiration

- [Matt Farmer - Codex model routing](https://mattfarmer.ai/codex-model-routing)
<https://mattfarmer.ai/codex-model-routing>

VERSION

2.0.0

A model-routing policy, validation harness, and evidence workflow for Claude Code project agents.

EXPLICIT

AUDITABLE

PORTABLE